Ders 9 Genişletilmiş Notu: Two-Way Association, Aggregation, Composition ve Kalıtım: Override, Polymorphism, super

Ders 9 Genişletilmiş Notu: Two-Way Association, Aggregation, Composition ve Kalıtım: Override, Polymorphism, super

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kod Yazmadan Öğrenilmez
- 2. Geçen Dersin Tekrarı: Beş İlişki Türü
- 3. Two-Way Association
- 4. Aggregation: Gevşek Parça-Bütün İlişkisi
- 5. Composition: Sıkı Parça-Bütün İlişkisi
- 6. Galeri Örneği: Dizilerle Çokluk
- 7. Inheritance (Kalıtım): Temel Kavramlar
- 8. Override, final ve Kalıtım Ağacı
- 9. Polymorphism ve Yerine Geçebilirlik
- 10. Kalıtımın Yanlış Kullanımı: GUI Düğmesi
- 11. Kalıtımı Ne Zaman Seçmeli?
- 12. Dynamic Binding
- 13. extends ve super: Person, Employee, Manager
- 14. Overriding ve Overloading Farkı
- 15. Kalıtımın Yanlış Kullanımı: Hasta ve Doktor
- 16. java.lang.Object ve toString()

1. Kod Yazmadan Öğrenilmez

Hoca derse başlamadan önce önemli bir uyarı yaptı: Yalnızca izleyerek nesneye yönelik programlama öğrenilemez, kod yazmak gerekir. C'den farklı olarak burada IDE'yi de kullanmak gerekir. Get ve set metotları, yapıcılar (constructor) ve `toString()` gibi tekrar eden kısımlar IDE'ye otomatik oluşturtulmalı, böylece asıl tasarıma zaman kalır.

> **Hocanın vurgusu:** Sadece izleyerek nesneye yönelik programlama konusunda bir şeyler öğreneceğinizi sanıyorsanız yanılıyorsunuz. Kod yazmak gerekiyor.

2. Geçen Dersin Tekrarı: Beş İlişki Türü

Şimdiye kadar nesneler arasındaki ilişkileri inceledik. Toplam beş ilişki türü vardır ve bunlar iki düzeye ayrılır:

- **Nesne düzeyinde ilişkiler:** association, dependency, aggregation, composition.
- **Sınıf düzeyinde ilişki:** inheritance.

2.1 Association ve Dependency

Association bir sahiplik ilişkisidir: "A, B'ye sahiptir." Bu ilişkide yön önemlidir, ok ilişkinin yönünü gösterir. İlişkinin üzerine kardinalite de eklenebilir, örneğin bir, çok ya da * (sıfır veya daha çok). Hocaya göre diyagramda gizli bilgi de bulunabilir. Bu yüzden ilişkinin özünü çıkarmamız, gösterimden çıkarım yapmamız gerekir.

Dependency ile association arasındaki fark, nesnenin nerede kullanıldığıdır. Association'da nesne, sınıfın bir üye değişkenidir (member). Dependency'de ise nesne yalnızca belirli bir metodun içinde kullanılır, herhangi bir sınıfın üye değişkeni değildir.

2.2 Sequence Diagram

Hoca geçen dersin tek yönlü association örneğinin sequence diagram'ını bir kez daha gösterdi. Okunuşu şöyledir:

1. Java sanal makinesi `main` metodunu çağırır.
2. `main` içinde `Person` sınıfından bir nesne üretilir (`octay`). Bu bir istektir. İsteğin içinde yapıcı çalışır, yanıt olarak nesnenin referansı döner.
3. `setCar` çağrılır. Bu da bir istektir, sıradan bir metot çağrısıdır.
4. `introduceSelf` çağrılır.
5. `Person` sınıfından ikinci bir nesne üretilir. İstek ve yanıt yine `main` ile nesne arasındadır.
6. İkinci nesne için `introduceSelf` çağrılır.

> **Hocanın vurgusu:** Sequence diagram çok önemli. Sınavda "bu senaryonun sequence diagram'ını çizin" diye sorulabilir, bu gösterimi öğrenin.

3. Two-Way Association

Two-way association'da her iki sınıf da diğerinin referansını saklar. `Person` sınıfının içinde bir `Car`, `Car` sınıfının içinde de `owner` adında bir `Person` tutulur. Böylece kişiden yola çıkıp arabasının bilgisine, arabadan yola çıkıp sahibinin bilgisine ulaşabiliriz. Hoca bu yapının özellikle veri tabanı benzeri uygulamalarda çok karşımıza çıktığını, nesneye yönelik programlamada ise referanslarla kurulduğunu söyledi.

3.1 Kardinalite: 0..1

İlişkinin iki ucunda da `0..1` yazar. Bir kişinin arabası olmayabilir, bir arabanın da sahibi olmayabilir. Hoca, birazdan göreceğimiz daha güçlü ilişki modelinde sıfırın olamayacağını, en az bir olması gerektiğini önceden belirtti.

> **Ders sonrası not:** Slayt `0..1` için "0 veya daha fazla kişi" diyor, hoca da "hiç sahibi olmayabilir veya en az bir sahibi olabilir" diye açıkladı. Doğrusu, `0..1` sıfır ya da en çok bir demektir. Aşağıdaki `setOwner` de tek parametre alır ve tek bir `owner` alanına yazar.

3.2 Tek Yönlüden İki Yönlüye Geçiş

Tek yönlü association'da `Person` içinde bir `Car` referansı vardı, `Car` içinde ise `Person` yoktu. İki yönlü hâle getirince yeni referans eklemek gerektiği için `Car` sınıfı değişti, `Person` aynı kaldı. Sınıfın değişmesi yüzünden örnek ayrı bir pakete konmuştur. Mevcut tasarımda bir `Car` nesnesi sahipsiz de oluşturulabilir.

`Car` sınıfına `owner` eklenince `introduceSelf` metodu da değişmek zorundadır. Arabanın sahibi varsa `owner.getName()` gibi bir çağrıyla sahibinin bilgisi alınır. Sahibi yoksa önce bir kontrol gerekir. Bu kontrol yapılmazsa geçen derste gördüğümüz `NullPointerException` alınır. Hocanın gerekçesi şudur: Sınıfın içine referans tanımı yazmak nesne üretmek değildir. Nesne ancak `new` ile üretilip referans atandıktan sonra vardır.

Aşağıdaki kod anlatılan yapıyla yazıldı ve javac ile denendi. Slayttaki metin ve plakalar altyazıda okunmadığı için çıktı metinleri ve nesne adları bana aittir. Kod, dersteki tartışmanın adlarıyla (`P1`, `P2`, `C1`, `C2`) kuruldu. Tek dosyada derlenebilsin diye sınıflar `public` yazılmadı.

```java
class Person {
    private String name;
    private Car car;              // tek yönlü sürümde de vardı

    public Person(String name) { this.name = name; }
    public String getName() { return name; }
    public Car getCar() { return car; }
    public void setCar(Car car) { this.car = car; }

    public void introduceSelf() {
        if (car != null)
            System.out.println(name + ", arabası: " + car.getPlate());
        else
            System.out.println(name + ", arabası yok");
    }
}

class Car {
    private String plate;
    private Person owner;         // iki yönlü olunca eklenen referans

    public Car(String plate) { this.plate = plate; }
    public String getPlate() { return plate; }
    public Person getOwner() { return owner; }
    public void setOwner(Person owner) { this.owner = owner; }

    public void introduceSelf() {
        if (owner != null)        // kontrol yoksa NullPointerException
            System.out.println(plate + ", sahibi: " + owner.getName());
        else
            System.out.println(plate + ", sahibi yok");
    }
}

public class IkiYonluDeneme {
    public static void main(String[] args) {
        Person p1 = new Person("P1");
        Person p2 = new Person("P2");
        Car c1 = new Car("C1");
        Car c2 = new Car("C2");

        c1.introduceSelf();       // henüz atama yok
        try {
            System.out.println(c1.getOwner().getName());
        } catch (NullPointerException e) {
            System.out.println("Atama yok: NullPointerException");
        }

        p1.setCar(c1);  c1.setOwner(p1);
        p2.setCar(c2);  c2.setOwner(p2);

        c1.setOwner(p2);          // araba el değiştirdi
        p1.introduceSelf();       // P1 hâlâ C1'i gösterir
        p2.introduceSelf();       // P2 hâlâ C2'yi gösterir
        c1.introduceSelf();       // C1'in sahibi artık P2
        c2.introduceSelf();       // C2'nin sahibi de P2
    }
}
```

Çalıştırınca çıktı şudur:

```
C1, sahibi yok
Atama yok: NullPointerException
P1, arabası: C1
P2, arabası: C2
C1, sahibi: P2
C2, sahibi: P2
```

3.3 İki Yönü de Atamak Zorunludur

Hoca slayttaki `main` programında 6. ve 7. satırları gösterip bunların neden yazıldığını sordu. Bu satırlarda kişiye araba, arabaya da sahip atanıyordu.

Soru: Bu iki satırı neden yazmak zorunda kaldık? Alanlar private olduğu için mi? **Cevap:** Hayır, diğer alanlar da private. Asıl sebep ilişkinin kendisi. Kişi sınıfı bir `Car` referansı, araba sınıfı da bir `Person` referansı tutmak zorundadır. Two-way association ancak bu atamalar yapılırsa sağlanır. Yapılmazsa referans `null` kalır ve `NullPointerException` alınır.

İki yön kurulunca plakası verilen bir arabanın sahibinin bilgilerine, adı verilen kişinin arabasının bilgilerine erişebiliriz.

3.4 Atamaları Çaprazlarsak

Hoca bir de şunu sordu: Birinci kişiye ikinci arabayı, ikinci kişiye yine ikinci arabayı atasak ne olur? Yani `P1` ve `P2` aynı `C2` nesnesini gösterebilir mi?

Soru: `P1` de `P2` de aynı arabayı gösterebilir mi? **Cevap:** Evet, atama yapılabilir, hiçbir sıkıntı çıkmaz. Bu şekilde many-to-many gibi bir görünüm oluşur. Ama arabanın kendi tarafına bakarsak `setOwner` tek parametre alır. Yani sabit bir `C2` arabasına aynı anda en fazla bir sahip atanabilir.

Soru: `C2.setOwner(P1)` dedikten sonra `C2.setOwner(P2)` dersek ne olur? **Cevap:** Araba el değiştirmiş olur. Eski bağ gider, araba artık başka kişiyi gösterir.

Bir öğrenci aynı durumu ilk bağlantıları kurulmuş nesneler üzerinden sordu: Başlangıçta `P1` ile `C1`, `P2` ile `C2` birbirini gösteriyor. Sonra `C1`'in sahibi `P2` yapılıyor.

Soru: O zaman ilk kullanıcının arabası `null` mı görünecek? **Cevap:** Hayır. `P1` üzerinde hiçbir atama yapılmadı, o yüzden `P1` hâlâ `C1`'i gösterir. `C1`'in `owner` alanı `P2`'ye döner, `C2` de zaten `P2`'yi gösteriyordu. Bir kişinin birden çok arabası varmış gibi olur, bu kendi başına sorun değildir.

Son durum, yukarıdaki programın çıktısıyla da aynıdır:

Nesne	Alan	Başlangıç	`c1.setOwner(p2)` sonrası
`P1`	`car`	`C1`	`C1` (değişmedi)
`P2`	`car`	`C2`	`C2` (değişmedi)
`C1`	`owner`	`P1`	`P2`
`C2`	`owner`	`P2`	`P2` (değişmedi)

Görüldüğü gibi `P1`, `C1`'i gösteriyor ama `C1`'in sahibi `P1` değil. İki yön birbiriyle tutarsız hâle geldi.

3.5 İlişkiyi Güçlendirmek

Hocaya göre bu kusurların hepsi two-way association'ı daha kuvvetli kurarak giderilebilir. En basit yöntem `this` referansını kullanmaktır. `this`, nesnenin kendi referansıdır. Örneğin `this.id` yazmak, fiziksel olarak oluşmuş bu nesnenin `id` alanına erişmektir. `setCar` içinde gelen arabanın sahibine bakılır, sahip `this`'e eşit değilse atama yapılır. Aynısı `Car` tarafında, `owner.getCar()` ile yapılır.

Slaytta gösterilen çözüm biraz farklıdır. `Car` yapıcısı plakayı doğrudan atar, sahibi ise parametre olarak alıp `setOwner`'a gönderir. Böylece nesne üretilirken dolaylı yoldan atama yapılmış

olur. Kişi nesnesine araba için ayrıca bir şey gönderilmez, araba yapıcısına kişi nesnesinin referansı verilir, iki taraf birbirini görür. İki yöntem aşağıda birlikte yazıldı ve javac ile denendi.

```
class Person {
    private String name;
    private Car car;

    public Person(String name) { this.name = name; }
    public String getName() { return name; }
    public Car getCar() { return car; }

    public void setCar(Car car) {
        this.car = car;
        if (car != null && car.getOwner() != this)
            car.setOwner(this);
    }
}

class Car {
    private String plate;
    private Person owner;

    public Car(String plate, Person owner) {
        this.plate = plate;
        setOwner(owner);     // nesne oluşurken iki yön de kurulur
    }
    public String getPlate() { return plate; }
    public Person getOwner() { return owner; }

    public void setOwner(Person owner) {
        this.owner = owner;
        if (owner != null && owner.getCar() != this)
            owner.setCar(this);
    }
}

public class KorumaliDeneme {
    public static void main(String[] args) {
        Person octay = new Person("Oktay Sinanoğlu");
        Car rover = new Car("PLAKA-ROVER", octay);
        System.out.println(octay.getCar().getPlate());
        System.out.println(rover.getOwner().getName());

        Person yeni = new Person("Yeni sahip");
        rover.setOwner(yeni);     // el değiştirme
        System.out.println(rover.getOwner().getName());
        System.out.println(yeni.getCar().getPlate());
        System.out.println(octay.getCar().getPlate()); // eski bağ duruyor
    }
}
```

```
PLAKA-ROVER
Oktay Sinanoğlu
Yeni sahip
PLAKA-ROVER
PLAKA-ROVER
```

> **Ders sonrası not:** Dersteki `this` kontrolü iki yönü kurmayı garantiler, ama eski bağı kendiliğinden koparmaz. javac ile denendi: `rover.setOwner(yeni)` sonrasında arabanın sahibi yeni kişi oldu, eski kişinin `car` alanı ise hâlâ aynı arabayı gösterdi (çıktının son satırı). Bu durumu da gidermek için `setOwner` içinde eski sahibin refe-

ransını ayrıca temizlemek gerekir.

Hoca karşılaştırmayı şöyle bağladı: İki yönlü ilişkide nesnelerin birbirini referans etmesi kolaylık sağlar, ama olağan dışı durumlar için ek kontroller gerekir. Tek yönlü ilişkide yalnızca bir taraf referans tuttuğu için karşı taraftan karışma ihtimali yoktur, kardinalite `0..1` ise ikinci bir nesne atama imkânı da olmaz. Dolayısıyla tek yönlü ilişki daha güvenlidir.

4. Aggregation: Gevşek Parça-Bütün İlişkisi

Aggregation bir bütün-parça (whole-part) ilişkisidir. İçi boş elmasla gösterilir. Örnek: Bir otobüs hattında (bus route) en az bir durak (stop) vardır. Burada sahiplik sıfır olamaz, en az bir nesne sınıfın içinde tutulmak zorundadır. Bu, kişinin arabası olmayabildiği association'dan farklıdır. Diyagramda kardinalite yazılmıyorsa 1 kabul edilir, bu biraz gizli bir bilgidir. Çokluğu göstermek için * kullanılır.

> **Hocanın vurgusu:** Aggregation association'dan güçlüdür, ama yalnızca kavramsal olarak. Kodlama bakımından arada güçlü bir fark yoktur.

Bütün ile parça arasında katı bir bağ yoktur, ilişki esnektir. `A` örneğinin birden çok `B` örneği vardır, `A` bütün, `B` parçadır. İkisi arasındaki ilişki silinse bile iki nesne de yaşamaya devam edebilir.

Bunu koddan değil senaryodan çıkarırız. Hocanın örneği öğrencilerin kampüse geldiği 41AT hattıdır. 41AT'yi silince ona bağlı bütün durakları da silseydik, aynı duraklardan geçen başka hatlar da etkilenirdi, örneğin diğer hatların üzerinde Eyüp durağı görünmezdi. Oysa bir durak yalnızca bir hatta ait değildir. Bu yüzden hat ve duraklar birbirinden bağımsız yaşamalıdır.

İkinci örnek öğrenci ve derstir. Öğrenci en az bir, en fazla beş derse kayıt olur, kardinalite bir ila beştir. Öğrenci dersten çekildiğinde ders silinmemelidir. Bu ilişki de aggregation ile kurulur. Hoca, senaryoları okuyup anlamayı ve geçen haftaki veri tabanı örneğindeki gibi düşünmeyi önerdi.

5. Composition: Sıkı Parça-Bütün İlişkisi

Composition iki nesne arasındaki katı bütün-parça ilişkisidir. Burada da sayı yazılmamışsa bir kabul edilir, diğer uçta * olabilir. Composition'ın aggregation'a göre gücü şudur: Parça aynı anda yalnızca bir bütüne ait olabilir.

Slaytta takım ve oyuncu örneği vardı: Takım bir dizi oyuncudan oluşur. Hoca bu örneği pek iyi bulmadı, çünkü oyuncu transfer olabilir, takım ile oyuncu aynı anda yaşayabilir. Ama slayttaki tanımı bir kod ayrımı olarak açıkladı:

- **Composition:** `Team` sınıfının yapıcısında `new Player` yapılır. Bu yüzden oyuncu başka bir takıma atanamaz, takım silinince yapıcıda oluşan nesneler de silinir.
- **Aggregation:** `BusRoute` yapıcısında `Stop` nesnesi oluşturulmaz. `Stop` ayrı, `BusRoute` ayrı oluşturulur. Sonra araba örneğindeki gibi birbirine `set` edilir ya da yapıcı yardımıyla dışarıdan atanır. Bütünün içinde `new` kullanılmaz.

  Ders sonrası not: Hoca "takım nesnesi silinince yapıcısında oluşan nesneler de silinir" dedi. Java'da bu otomatik bir bağ değildir. javac ile denendi: Yapıcıda `new` ile oluşan parça bir get metoduyla dışarıya verildi, bütünün referansına `null` atandı, parça yine kullanılabildi. Composition'ın "parça bütünle birlikte yok olur" kuralı bu yüzden tasarımın sağlaması gereken bir kuraldır. Parçanın referansı dışarı verilmemelidir.

Hoca farkı bir insan ve bir kütüphane örneğiyle de anlattı. İnsan vefat edince organları da yok olur, bu composition'dır. Bir kütüphaneyi yok ettiğinizde raflar yok olabilir, ama kitaplar yok olmak zorunda değildir.

5.1 Üç İlişkinin Kodlanması

Association, aggregation ve composition'ın kodlanması benzerdir. Çokluk (multiplicity) için diziler ya da veri yapıları kullanılır: liste, kuyruk, yığın, bağlı liste, ağaç, graf, heap ya da dizi. Veri yapıları dersinde gördüklerimizin aynısı burada da kullanılır, bu konu iki hafta içinde

ayrıca ele alınacaktır. Hoca önce nesne dizileriyle başlıyor. Daha önce gördüğümüz kitap ve yazar dizileri bunun örneğidir, `float` ya da `int` dizisine benzer.

6. Galeri Örneği: Dizilerle Çokluk

Slaytta bir `Gallery` sınıfı isteniyor. Galeri birden çok `Car` ile ilişkilidir ve bu ilişki dizi ile kurulacaktır. Örnek, derste yaptığımız kitap örneğine çok benzer. Slaytın yazarı galeri ile araba arasındaki ilişkiyi composition olarak göstermeyi seçtiğini, bir ve * kardinaliteli bir association olarak da gösterilebileceğini belirtiyor. Daha az parametreli bir yapıcı eklemek de istenebilirdi, ama galeriye geçici sahip olarak bir `Person` atanmasının tercih edildiği yazıyor.

6.1 Sınıf Şemasını Okumak

Üç sınıf arasındaki ilişkiler şöyledir:

- **`Person - Car`:** Two-way association.
- **`Gallery - Car`:** Composition. Galeride birden çok araba vardır ve bunlar dizi ile tutulur.
- **`Gallery - Person`:** `galleryOwner` alanı üzerinden one-way association.

Hoca bu ilişkilerin türünü öğrencilere tek tek sordu.

Soru: Galeri ile araba arasındaki ilişki nedir? **Cevap:** Görünüşe bakılırsa composition da aggregation da olabilir. Galerideki arabalar galerinin parçasıdır. Galeriyi yok edince arabalar da yok olacaksa composition ağır basar. Aggregation ile composition arasındaki küçücük fark burada ortaya çıkar.

Soru: Galeri ile galeri sahibi arasındaki ilişki nedir? Bir öğrencinin cevabı: Galeriyi yok edince kişi yok olmaz. **Cevap:** Composition böylece elendi. Ama daha temel bir soru var: Aralarında parça-bütün ilişkisi var mı? Kişi galerinin bir parçası değildir, galeride insan satılmıyor. Parça-bütün olmadığı için aggregation da değildir. Bu bir yönetme ilişkisidir, galerinin bir sahibi vardır. Okun yönü de tek olduğu için one-way association'dır.

Soru: Kişi ile araba arasındaki ilişki nedir? Bir öğrencinin cevabı: Parça-bütün yok, bu yüzden aggregation ve composition olmaz. Bir bağımlılık da görünmediği için dependency de değildir, association olur. **Cevap:** Doğru. Kişinin arabaya, arabanın kişiye sahipliği vardır. İki yönlü olduğu için two-way association'dır.

> **Hocanın vurgusu:** Bu analizi UML sınıf diyagramında yapabilmeniz lazım. Bundan sonra kodlaması çok kolaydır. İlişkilerin türünü görmeden körlemesine yazarsanız olmaz.

Şemadaki metotlar da aynı mantıkla okunur: üyeler için get ve set metotları, galeriye araba ekleme, iki türlü arama (nesneyle ve plakayla), silme, satma ve galeri bilgilerini yazan `introduceSelf`. Araba satışı, arabanın `owner` değerini değiştirmek demektir.

6.2 Alanlar ve Yapıcı

Galerinin alanları şunlardır: ad, araba dizisi, bir tamsayı (araba sayısı) ve galeri sahibi. Yapıcıda ad ve sahip atanır, araba sayısı sıfırdan başlatılır, dizi oluşturulur.

Dizinin oluşturulduğu satırı hoca `new` kullanıldığı için composition'ın göstergesi olarak işaretledi. Bu bir bellek ayırmadır (memory allocation). Burada en fazla 30 arabanın nesnesi değil, referansı tutulur. Atamalar daha sonra yapılır.

Dizinin boyutunu `cars.length` ile öğrenebiliriz, ama bu dolu eleman sayısını vermez, üst sınırı verir. Bu yüzden C'deki `#define` gibi ayrı bir üst sınır değişkeni gerekmez. Etkin araba sayısı için ise `carCount` gibi bir değişken gerekir. Sınır yine de bir sabit olarak yazılabilir:

```
public final static int MAX_CARS = 30;
```

Bu değer değiştirilemez ve bütün nesneler onu 30 olarak görür. javac ile denendi: Sabite yeni değer atamaya çalışmak `cannot assign a value to static final variable` hatası verir.

> **Ders sonrası not:** Hoca `new` ile dizi oluşturulduğu için ilişkinin composition olduğunu söyledi, ama aynı derste composition için "parçalar bütünün içinde üretilir"

ölçütünü de verdi. javac ile denendi: `new Car[30]` yalnızca 30 tane `null` referans ayırır, `Car` nesnesi üretmez. Testte arabalar `main` içinde üretilip galeriye veriliyor, satılan araba da galeriden çıktıktan sonra yaşamaya devam ediyor. Kod olarak bu yapı aggregation ölçütüne daha yakındır. Hocanın dediği gibi burada composition kararı senaryodan gelir.

6.3 Araba Ekleme

Galeriye araba eklerken önce arabanın galeride olup olmadığına bakılır. Araba yoksa ve galeri dolu değilse (`carCount < cars.length`) araba diziye eklenir, `carCount` bir artar, `true` döner. Bu şartlardan biri sağlanmıyorsa `false` döner. Kitap örneğinin neredeyse aynısıdır.

6.4 Arama ve Method Overloading

Ekleme metodunu yazabilmek için önce arama metodu gerekir. Slayttaki iki arama metodu, kısa nottaki hâliyle, aşağıdaki kodun içindedir. İlkinde `for-each` döngüsüyle dizinin bütün elemanları gezilir, gelen nesneye eşit olan varsa `true`, yoksa `false` döner. Burada aranan şey nesnenin kendisi, yani adresidir. Bir yerde oluşturulan `Car` nesnesinin referansı galeriye atanmıştır, gelen parametre de bu referanstır. Bu adres dizinin içindeyse araba daha önce eklenmiş demektir.

İkincisinde sıfırdan `carCount`'a kadar normal `for` döngüsü kurulur. Her arabanın plakası alınır, gelen metinle `compareTo` metoduyla karşılaştırılır. Sonuç sıfırsa nesnenin kendisi döner, bulunamazsa `null` döner.

> **Ek bilgi (derste anlatılmadı):** Referans eşitliği (==) ile `compareTo` farklı şeyleri sınar. javac ile denendi: Aynı metni taşıyan iki ayrı `new String` nesnesi için == sonucu `false`, `compareTo` sonucu `0` çıktı. Plakayı metin olarak aramak için bu yüzden `compareTo` kullanılır.

Bu iki metodun adı aynı, parametreleri farklıdır. Buna **method overloading** denir. Dönüş tipleri de farklıdır, ama overloading için önemli olan parametrelerin farklı olmasıdır.

> **Ders sonrası not:** Altyazıda hocanın sözü "şunları farklı yazarsak hata alırız" diye geçiyor. Bağlamdan kastedilen, parametreleri aynı yazmaktır. javac ile denendi: Yalnızca dönüş tipi farklı iki `searchCar(String)` yazılınca `method searchCar(String) is already defined` hatası çıkıyor.

6.5 Silme: Kaydırma İşlemi

Bir nesneyi listeden silmek için önce onu aramak gerekir. Bulununca silinir, sonra kaydırma (shift) yapılır, çünkü bu statik bir dizidir ve C'deki diziyle aynı davranır. Hocaya göre bağlı liste kullanılsaydı kaydırmaya gerek kalmazdı.

Adımlar şöyledir:

1. Sıfırdan `carCount`'a kadar arabaların plakası alınır ve aranan plakayla karşılaştırılır.
2. Eşitlik varsa `cars[i]` geçici bir değişkende saklanır.
3. Sonraki arabalar birer sola kaydırılır.
4. Kaydırma sonrasında son değer iki kez bulunur. Bu tekrarı gidermek için son konum `null` yapılır.
5. `carCount` bir azaltılır.
6. Silinen araba, kullanıcıya bilgi vermek için döndürülür.
7. Araba bulunamazsa `null` döner.

Silinen nesne yaşayan bir nesnedir, `cars` dizisi yalnızca nesnelerin adreslerini saklar. Bu yüzden ne kadar kaydırırsak kaydıralım en sondaki adres yinelenir, onu temizleyip sayacı azaltmak iyi bir kontroldür.

> **Ders sonrası not:** Kısa nottaki slayt kodunda iç döngü `for (int j = i; j < carCount; j++)` biçimindedir. javac ile denendi: 30 kapasiteli dizide 29 araba varken silme çalıştı, dizi doluyken (`carCount` 30 iken) `ArrayIndexOutOfBoundsException: Index 30 out of bounds for length 30` hatası çıktı. Doğrusu `j < carCount - 1`'dir. Aşağıdaki kod bu düzeltmeyle yazıldı.

6.6 Satma

`sellCar`, plakayı ve yeni sahibi parametre olarak alır. Hoca burada terim düzeltmesi de yaptı: Metot tanımındakilere parametre, metot çağrısındakilere argüman denir. Önce `removeCar` denenir. Araba dizideyse silme başarılıdır ve dönen nesne satılan arabadır. Sonuç `null` değilse arabanın sahibi yeni kişi yapılır ve `true` döner. Aksi hâlde `false` döner.

6.7 Test Sınıfı

Testte dikkat çeken ilk şey yapıcı çağrısıdır: `new Gallery("Cars for You", new Person(...))`. Hoca bazı öğrencilerin "new kullanmış ama eşittir yok" diye şaşırdığını söyledi. Eşittir kullanmak zorunlu değildir. Yapıcıya `Person` türünde bir nesne gelecektir, bunun önceden atanmış bir referanstan gelmesi gerekmez. Önce kişi oluşur, oluşan kişi galeri yapıcısına gönderilir.

Sonra iki araba oluşturulur, ikisinde de ikinci parametre `null`'dır, yani sahipleri yoktur. Arabalar galeriye eklenir ve işlemin başarısı yazılır. BMW aranır. Audi'nin plakası `getPlate` ile alınır, `removeCar` çağrılır, dönen nesne Audi nesnesiyle karşılaştırılır, çünkü metot nesne döndürür. Sonra Audi yeniden aranır.

Aşağıda sınıfların tamamı ve test programı vardır. Dersteki anlatıma göre yazıldı, javac ile derlenip çalıştırıldı. Plakalar ve kişi adları altyazıda olmadığı için yer tutucudur. Dersin test akışına tekrar ekleme ve satış satırları eklendi.

```
class Person {
    private String name;
    private Car car;

    public Person(String name) { this.name = name; }
    public String getName() { return name; }
    public void setCar(Car car) { this.car = car; }
}

class Car {
    private String plate;
    private Person owner;

    public Car(String plate, Person owner) {
        this.plate = plate;
        setOwner(owner);
    }
    public String getPlate() { return plate; }
    public Person getOwner() { return owner; }

    public void setOwner(Person owner) {
        this.owner = owner;
        if (owner != null)
            owner.setCar(this);
    }

    public void introduceSelf() {
        String sahip = (owner != null) ? owner.getName() : "yok";
        System.out.println(plate + ", sahibi: " + sahip);
    }
}

class Gallery {
    public final static int MAX_CARS = 30;
    private String name;
    private Car[] cars;
    private int carCount;
    private Person owner;
```

```java
public Gallery(String name, Person owner) {
    this.name = name;
    this.owner = owner;
    this.carCount = 0;
    this.cars = new Car[MAX_CARS]; // 30 araba değil, 30 boş referans
}

public boolean addCar(Car aCar) {
    if (!searchCar(aCar) && carCount < cars.length) {
        cars[carCount] = aCar;
        carCount++;
        return true;
    }
    return false;
}

public boolean searchCar(Car aCar) {
    for (Car car : cars)
        if (car == aCar)
            return true;
    return false;
}

public Car searchCar(String plate) {
    for (int i = 0; i < carCount; i++)
        if (cars[i].getPlate().compareTo(plate) == 0)
            return cars[i];
    return null;
}

public Car removeCar(String plate) {
    for (int i = 0; i < carCount; i++) {
        if (cars[i].getPlate().compareTo(plate) == 0) {
            Car theCar = cars[i];
            for (int j = i; j < carCount - 1; j++) // düzeltildi
                cars[j] = cars[j + 1];
            cars[carCount - 1] = null;
            carCount--;
            return theCar;
        }
    }
    return null;
}

public boolean sellCar(String plate, Person newOwner) {
    Car soldCar = removeCar(plate);
    if (soldCar != null) {
        soldCar.setOwner(newOwner);
        return true;
    }
    return false;
}

public void introduceSelf() {
    String mesaj = name + " galerisi, sahibi: " + owner.getName()
            + ", araba sayısı: " + carCount;
    System.out.println(mesaj);
    for (int i = 0; i < carCount; i++)
        cars[i].introduceSelf();
}
```

```
}

public class GaleriTesti {
    public static void main(String[] args) {
        Gallery carsForYou =
                new Gallery("Cars for You", new Person("Galeri sahibi"));
        Car bmw = new Car("PLAKA-BMW", null);   // henüz sahibi yok
        Car audi = new Car("PLAKA-AUDI", null);

        System.out.println("BMW ekleme: " + carsForYou.addCar(bmw));
        System.out.println("Audi ekleme: " + carsForYou.addCar(audi));
        System.out.println("BMW tekrar: " + carsForYou.addCar(bmw));
        carsForYou.introduceSelf();

        System.out.println("BMW arama: " + carsForYou.searchCar(bmw));
        Car silinen = carsForYou.removeCar(audi.getPlate());
        System.out.println("Audi silme: " + (silinen == audi));
        System.out.println("Audi arama: " + carsForYou.searchCar(audi));

        Person alici = new Person("Alıcı");
        System.out.println("BMW satış: "
                + carsForYou.sellCar("PLAKA-BMW", alici));
        bmw.introduceSelf();      // galeriden çıktı ama yaşıyor
        carsForYou.introduceSelf();
    }
}
```

```
BMW ekleme: true
Audi ekleme: true
BMW tekrar: false
Cars for You galerisi, sahibi: Galeri sahibi, araba sayısı: 2
PLAKA-BMW, sahibi: yok
PLAKA-AUDI, sahibi: yok
BMW arama: true
Audi silme: true
Audi arama: false
BMW satış: true
PLAKA-BMW, sahibi: Alıcı
Cars for You galerisi, sahibi: Galeri sahibi, araba sayısı: 0
```

> **Ders sonrası not:** `searchCar(Car)` içindeki `for-each` dizinin boş `null` konumlarını da gezer. javac ile denendi: Dizide boş yer varken `searchCar(null)` çağrısı `true`, dizi doluyken `false` döndü. Döngüyü `carCount`'a kadar kurmak bu sorunu önler.

Bu bölümün sonunda hoca nesne düzeyindeki ilişkileri topladı: association, dependency, aggregation, composition. Association, aggregation ve composition'ı katılık sırasına koydu, dependency'nin bunlardan biraz farklı olduğunu söyledi. Bunlar nesne üretince nesneler arasında kurulan ilişkilerdir. Şimdi sınıf düzeyindeki ilişkiye geçiyoruz.

7. Inheritance (Kalıtım): Temel Kavramlar

Hocaya göre kalıtım dönemin, nesneye yönelik programlamanın en önemli konularından biridir. Gösterimi içi boş (beyaz) üçgendir. Okun yönü önemlidir: Ok alt sınıftan üst sınıfa doğrudur.

Terimlerin hepsi kullanılır:

- **`A` üst sınıf:** parent class, superclass, base class.
- **`B` alt sınıf:** child class, subclass, derived class.

Gerçek dünyada bir çocuk genetik özellikleri anne babasından alır. Kalıtım, var olan bir sınıftan yeni bir sınıf oluşturmanın buna benzer bir yoludur. Bu bir sınıf düzeyi ilişkidir, nesne düzeyi değildir.

7.1 protected ve Erişim

A'nın özelliklerinden bazıları B'ye geçer. Hoca "bazıları" sözünü özellikle vurguladı, çünkü bütün üyeler erişilebilir olmaz. Slaytın anlatımı şudur: Private üyeler dışında bütün alan ve metotlara erişilebilir. Kalıtımla erişildiğini açıkça göstermenin makul yolu `private` ya da `public` yerine `protected` yazmaktır. `public` her sınıfa açıktır, bu yüzden hoca "public kullanıyorsak kalıtıma gerek yok" dedi. Bazı bilgileri kalıtım alan sınıflar dışındaki herkesten gizlemek istiyorsak `protected` kullanırız. A'da `protected` yazılanı B görür. Ama C sınıfıyla association, aggregation ya da composition kursak bile C o üyelere erişemez.

Soru: Torun sınıflar (grandchild) `protected` üyelere erişemez mi? **Cevap:** Erişir. Alt sınıfın alt sınıfı da üst sınıftaki `protected` üyeleri görür. Kalıtım ağacının dışındaki sınıf ise göremez.

Hoca IDE'de bir nesne üzerinde metot listesini açarak `getClass` gibi metotları gösterdi. Bu metotların hepsi `public` tanımlıdır. `toString` gibi olanlar `Object` sınıfından gelir, çünkü bütün sınıflar doğrudan `Object`'ten kalıtım alır.

> **Ders sonrası not:** Hoca, kalıtım ağacı dışındaki sınıfların `protected` üyelere erişemeyeceğini söyledi. Bu yalnız başka paketteki sınıflar için doğrudur. javac ile denendi: Aynı paketteki ilişkisiz `Komsu` sınıfı `protected` alana erişti, başka paketteki alt sınıf ve onun alt sınıfı erişti, başka paketteki ilişkisiz `Yabanci` sınıfı `x has protected access` hatası aldı. Ayrıca "public kullanırsak kalıtıma gerek yok" sözü erişim bakımındandır. Aşağıda `Employee`, `public` olan `getName`'i de kalıtımla alır. Kalıtım yalnızca erişim için kullanılmaz.

8. Override, final ve Kalıtım Ağacı

8.1 Kalıtımın Kuralları

Alt sınıf üst sınıftaki bir üyeyi reddedemez, silemez. Üst sınıfta ne özellik varsa alt sınıf hepsini sağlamak zorundadır. Ama kalıtılan metotların gövdesi değiştirilebilir. Buna **overriding** denir. Overload, bir metodun birden çok tanımıdır. Override ise yeniden tanımlamadır: Son tanım çalışır, öncekisi o nesne için geçersiz sayılır. Üst sınıftaki metodu alt sınıfta aynı prototiple yeniden yazmak gerekir.

8.2 Override Örneği

`Person` sınıfında `introduceYourself` "Ben insanım" yazar. `Retired` sınıfında aynı prototiple yeniden yazılan metot "Ben emekliyim" yazar. `Retired` nesnesi için `Person`'daki sürüm iptal olur, `Retired` içindeki çalışır. `Employee`, `Student`, `Director` ya da `PartTime` gibi metodu yeniden yazmayan sınıflar `Person`'daki sürümü kullanmaya devam eder.

Hocanın günlük hayat örneği İstanbulkart'tır. Karta öğrenci olduğunuzu kaydettirirseniz öğrenci tarifesinden para düşer. Hiçbir şey beyan etmeden kart alırsanız varsayılan, yani normal kullanıcı tarifesi uygulanır. Öğretmen, emekli gibi başka kategoriler de vardır. Herkes bir `Person`'dır, ama farklı davranışları olabilir. Bazı özellikleri kalıtımla üst sınıftan alır, bazılarını alt sınıfta kendimiz tanımlarız.

Yeni üyeler de alt sınıflara eklenebilir. `Person`'a eklenen bir üye ise alt sınıflara doğrudan kalıtılır.

8.3 final Metot

Bir metoda `final` eklenirse o metot override edilemez. Hoca bunu slaytta `public String final introduceSelf()` ve `protected final String` biçiminde okudu, sırayı hatırlamadığını da söyledi.

> **Ders sonrası not:** javac ile denendi: `public String final introduceSelf()` derlenmedi (`<identifier> expected`). `final`, dönüş tipinden önce gelmelidir. `public final String`, `final public String` ve `protected final String` derlendi. `final` metodu alt sınıfta yeniden yazmak ise `cannot override` hatası verdi.

8.4 Kalıtım Ağacı

Bir alt sınıf başka sınıfların üst sınıfı olabilir. `Employee`, `Person`'ın alt sınıfı, `Director` ve `PartTime`'ın üst sınıfıdır. Bu şekilde oluşan yapıya kalıtım hiyerarşisi ya da kalıtım ağacı denir. Bu çok seviyeli (multi-level) kalıtımdır, çoklu (multiple) kalıtım değildir.

Hoca ağacı çok derin yapmamayı tavsiye etti. En üst sınıfta yapılan bir değişiklik bütün alt sınıfları etkiler, derin bir tasarımın yönetimi zorlaşır. Benzetme ağaçtandır: Kök, ara düğümler ve yapraklar vardır, derin bir ağaç rüzgârla kolayca devrilebilir.

8.5 Kalıtımın Doğurduğu Sonuçlar

Tekrar edelim: Kalıtım sınıf düzeyinde bir ilişkidir, unutmayın. Kalıtım **is-a** ilişkisidir: Jeep bir arabadır, yarış arabası bir arabadır, lazer yazıcı bir yazıcıdır. Association'da **has** ilişkisi vardı, composition ve aggregation'da bütün-parça ilişkisi vardı. Kalıtım ise genelleme ve özelleştirme ilişkisidir.

9. Polymorphism ve Yerine Geçebilirlik

Kalıtımın sonucu olarak ortaya çıkan kavramlardan biri polymorphism'dir: Alt sınıflarda aynı işin farklı yollarla yapılması. Nokta vuruşlu yazıcı bir belgeyi bir şekilde basar, lazer yazıcı başka, mürekkep püskürtmeli yazıcı başka bir şekilde basar.

Belge yazdırmak genel bir kavramdır, ama fiziksel uygulaması her zaman aynı değildir. `Printer` bir şeyin yazılması gerektiğini söyler, örneğin `void print` der. Nasıl yazılacağı alt sınıfların içinde tanımlanır. Lazerle ilgili bir yazdırma işini `Printer` bilmez. Ama bunların hepsi birer yazıcı olduğu için hepsinde yazdırma mutlaka bir şekilde bulunur. Bu yapıyı kurmak kalıtımın bir özelliğidir.

Üst sınıf, alt sınıflarının daha genel ve daha az yetenekli sürümüdür. Kavramsal metot ve üyeler üst sınıfta durur. Alt sınıf daha özel ve daha yeteneklidir.

Hoca aynı fikri canlılar üzerinden de anlattı: İnsan ve ona benzeyen maymun ya da aynı aileden kedi, kaplan, aslan. Üst sınıfta ortak özellikleri, alt sınıflarda farklı davranışları vardır. Ortak özellikler kalıtımdan, alttaki özel davranış farkları da kalıtımın sonucu olan polymorphism'den gelir.

Substitutability (yerine geçebilirlik): Üst sınıfın beklendiği her yerde alt sınıf kullanılabilir. Hoca kelimenin söylemesinin zor olduğunu da ekledi. Yazıcı yerine lazer ya da mürekkep püskürtmeli yazıcı, araba yerine jeep ya da yarış arabası kullanılabilir. Bu yüzden kalıtıma is-a ilişkisi de denir.

> **Ders sonrası not:** Altyazıda 01:37:01'de alt sınıflar için de "less capability" geçiyor. Doğrusu, hemen sonra okunan slayt cümlesindeki gibi alt sınıfların daha yetenekli olduğudur.

10. Kalıtımın Yanlış Kullanımı: GUI Düğmesi

Slaytta `GuiButton` genel kavram, kırmızı, sarı ve mor düğmeler de alt sınıflar olarak gösteriliyor. Bu doğru değildir, çünkü mor bir GUI düğmesi değildir, bir renktir. Is-a ilişkisi tutmaz. Her sınıf arasına doğrudan kalıtım kuramayız, arkasında bir mantık bulunmalıdır.

Doğru modelde `GuiButton` bir sınıftır, renk onun bir özelliğidir, `RGB` de kırmızı, yeşil ve mavi alanlarına sahip ayrı bir sınıftır. Sınıf yapısı javac ile derlendi:

```
class RGB {                         // renk ayrı bir sınıf
    private int red, green, blue;

    public RGB(int red, int green, int blue) {
        this.red = red;
        this.green = green;
        this.blue = blue;
    }
}
```

```java
class GuiButton {                // kalıtım yok: buton renge sahiptir
    private RGB color;

    public void setColor(RGB color) { this.color = color; }
    public RGB getColor() { return color; }
}

public class GuiDeneme {
    public static void main(String[] args) {
        GuiButton buton = new GuiButton();
        System.out.println(buton.getColor());  // rengi olmayabilir
    }
}
```

Soru: `GuiButton` ile `RGB` arasındaki ilişki nedir? Bir öğrencinin cevabı: Tek yönlü association. **Cevap:** Doğru. Buradaki sahiplik has ilişkisidir, is-a değildir. Peki neden aggregation ya da composition değildir? Çünkü onlarda parça-bütün vardır. Otobüs hattı duraklardan oluşmak zorundadır, ama GUI düğmesinin renklerden oluştuğunu söyleyemeyiz. Düğmenin rengi özelliğidir, sahiplik parça-bütün sahipliği değil, kullanabilme sahipliğidir. Slaytta kardinalite bire bir olduğu için bir ölçüde aggregation denebilir, composition denmez. Ama aslında ilişki association'dır.

11. Kalıtımı Ne Zaman Seçmeli?

Kalıtım çalışma zamanında değiştirilemeyen sınıf düzeyi bir ilişkidir. Çalışma sırasında üst sınıfa yeni metot ya da özellik ekleyemezsiniz. Polymorphism onun doğal sonucudur.

Deneyimli nesneye yönelik modelciler kalıtımı yalnızca iki sınıf arasında açık bir genel-özel (gen-spec) ilişkisi varsa seçer. Emin değilseniz kalıtım kurmayın. Başka ilişkiyle karıştırıp hemen kalıtım kurmak sık yapılan bir hatadır.

> **Hocanın vurgusu:** İlk başlarda bu hatayı çok yapacaksınız, önünüze gelen her şeye kalıtım ilişkisi bulacaksınız. Önce is-a ilişkisini kontrol edin. Kişiden öğrenci ya da öğretmen türer, orada kalıtım olur. Ama düğmenin renkleriyle arasında hiyerarşik bir ilişki yoktur.

Aksi hâlde association, composition ya da aggregation kullanılır. Bunların kullanımı kalıtımdan daha yaygındır, nesneye yönelik tasarımda kalıtımı daha az göreceğiz.

Java'da bir sınıf birden çok sınıftan kalıtım alamaz. Örneğin `Person`'ın yanında `Worker` diye bir sınıf daha olsa ikisinden birden kalıtım alınamaz, biri kaldırılmalıdır. javac ile denendi: `extends P, W` yazmak derleme hatası verir. Çoklu kalıtım için interface kullanılır, bunu sonra göreceğiz.

Kalıtımın doğru kullanımı, nesneye yönelik programlamadan sonra gelen nesneye yönelik analiz ve tasarım konusunun ve tasarım kalıplarının önemli bir parçasıdır. Yaşayan bir yazılım üretmek istiyorsanız nerede kalıtım kullanılıp kullanılmayacağı önemlidir.

> **Hocanın vurgusu:** Yazılımda bir şeyi değiştirmeye çalıştığınızda baştan yazmak daha kolay geliyorsa kötü bir tasarım yapılmış demektir.

12. Dynamic Binding

`Person`, `Student`, `Employee` ve `Retired` sınıfları bir kalıtım ağacı oluştursun, `introduceSelf` metodu alt sınıflarda override edilsin. Üst sınıftaki gövde boş bile olabilir. Alt sınıfta bu metodun içine ek satırlar yazılabilir.

Şimdi `Person` türünde bir dizi düşünelim: `Person[] p = new Person[10]`. `p[0]` için `new Student()`, `p[1]` için `new Retired()` yazılır. Slayttaki parantezler yalnızca yapıcı çağrısını gösterir. Bu nesneler üst sınıf dizisinin ilgili indislerine atanır. Peki dizinin bütün elemanları için `introduceSelf` çağrılırsa ne olur? `p[0]` için `Student`'taki, `p[1]` için `Retired`'daki, `p[2]` bir `Employee` ise `Employee`'deki sürüm çalışır.

Metodun doğru sürümü çalışma zamanında çalıştırılır. Buna **dynamic binding** denir, bu mekanizmanın adı da polymorphism'dir. Öğrenci kendini numarası ve sınıfıyla, işçi ve emekli başka türlü tanıtır. Hoca bunun özellikle Java'da dynamic binding ile yapıldığını belirtti.

Aşağıdaki kod anlatılanlardan yazıldı ve javac ile denendi. Slayttaki gibi yalnızca `Retired` metodu override ediyor. `Student`, metodu override etseydi o da kendi metnini yazardı.

```
class Person {
    public void introduceYourself() {
        System.out.println("Ben insanım");
    }
}

class Student extends Person { }       // override yok
class Employee extends Person { }      // override yok

class Retired extends Person {
    @Override
    public void introduceYourself() {  // aynı imza: override
        System.out.println("Ben emekliyim");
    }
}

public class KalitimDeneme {
    public static void main(String[] args) {
        Person[] p = new Person[10];
        p[0] = new Student();
        p[1] = new Retired();
        p[2] = new Employee();
        for (Person kisi : p)
            if (kisi != null)          // kalan 7 konum null
                kisi.introduceYourself();
    }
}
```

```
Ben insanım
Ben emekliyim
Ben insanım
```

> **Ek bilgi (derste anlatılmadı):** Derleyici çağrının geçerli olup olmadığını referansın tipine göre denetler, hangi sürümün çalışacağı ise nesnenin kendisine göre belirlenir. javac ile denendi: `Employee` tipli bir referans `Manager` nesnesini gösterirken `setBonus` çağrısı `cannot find symbol` hatası verdi. Bu yüzden aşağıdaki `Company` programında bonus önce `Manager` tipli `boss` referansıyla atanır.

Override edilmiş bir metodun önceki sürümüne erişmek istersek bunu `super` üzerinden yapabiliriz. Metodun içinde önceki tanımı `super` ile çağırırız. Override'ı engellemek istersek yukarıda gördüğümüz gibi `final` ekleriz.

Dizide `null` olan konumlar için `if (kisi != null)` kontrolü gerekir. javac ile denendi: Kontrol olmadan `for-each` döngüsü boş konuma gelince `NullPointerException` verdi.

13. extends ve super: Person, Employee, Manager

13.1 Sınıf Şeması

Slayttaki UML şeması bir kalıtım ağacından ve onları kullanan bir sınıftan oluşuyor:

- **`Person`:** Genel sınıf. `name` alanı, adı atayan yapıcı ve `getName` metodu burada. Çalışan ve yönetici için ad ortak özellik olduğu için yukarıda durur.
- **`Employee` (`Person`'ın alt sınıfı):** Ek bilgi olarak `salary` alanı, `getSalary` ve `setSalary` metotları.

- **`Manager` (`Employee`'nin alt sınıfı):** Ek bilgi olarak `bonus` alanı ve `setBonus` metodu. `getSalary`'nin uygulaması farklıdır, yani override edilir.
- **`Company`:** Test (sürücü) sınıfı. `Employee` ve `Manager` nesnelerini doğrudan kullanır.

`Person` türünde nesne üretilebilir, ondan `Employee` ve `Manager` da üretilebilir. Ama maaş `Person` için kullanılmaz. Hoca şu soruyu ön plana çıkardı: İki yapıcı da ad ve maaş alıyor, peki ad nerede atanacak? Alt sınıfın içinde değil, `super` üzerinden üst sınıfta atanacak.

13.2 extends ve super

Kalıtım ilişkisini kurmak için `extends` anahtar kelimesiyle üst sınıfın adı yazılır. `name` private, yapıcı ve `getName` ise public'tir. Bu yüzden `Employee` yapıcıya ve `getName`'e erişebilir, ama `name`'e doğrudan erişemez. `name`'e erişim dolaylıdır. `Employee` yapıcısında ad `super(name)` ile üst sınıfa gönderilir, maaş ise kendi alanına atanır.

> **Hocanın vurgusu:** `super` çağrısını yazacaksak birinci satırda yazmak zorundayız, sonraki satırlara yazamayız.

Hiçbir şey yazmazsak varsayılan (default) yapıcı çağrılır. Bu görünmeyen, boş bir `super()` çağrısıdır. Önce o çalışır, sonra alt sınıf yapıcısına geri dönülür.

`Manager`, `Employee`'den türer. Yapıcısında üst sınıf yapıcısı çağrılır, `bonus` sıfır atanır. Maaşı hesaplamak için `salary` üyesine doğrudan erişemeyiz, çünkü `Employee`'de private tanımlıdır. Bu yüzden `super.getSalary()` ile değeri dolaylı yoldan alıp bonusu ekleriz.

Dersin kodu anlatıma göre aşağıda yazıldı, javac ile derlendi ve çalıştırıldı. Kısa notta slaytın başında `package nyp05;` satırı bulunur, burada tek dosyada denendiği için yazılmadı. Slaytta iki `Employee` nesnesinin adı ve maaşı altyazıda yoktur, bu yüzden maaşlar yer tutucu olarak `0` yazıldı.

```
class Person {
    private String name;

    public Person(String name) { this.name = name; }
    public String getName() { return name; }
}

class Employee extends Person {
    private int salary;

    public Employee(String name, int salary) {
        super(name);              // ilk satır; name burada private
        this.salary = salary;
    }
    public int getSalary() { return salary; }
    public void setSalary(int salary) { this.salary = salary; }
}

class Manager extends Employee {
    private int bonus;

    public Manager(String name, int salary) {
        super(name, salary);
        bonus = 0;
    }
    public Manager(String name, int salary, int bonus) { // overloading
        super(name, salary);
        this.bonus = bonus;
    }
    public void setBonus(int bonus) { this.bonus = bonus; }

    @Override
    public int getSalary() {      // overriding
```

```
        return super.getSalary() + bonus;
    }
}

public class Company {
    public static void main(String[] args) {
        Employee[] staff = new Employee[3];
        Manager boss = new Manager("Yönetici", 8000);
        boss.setBonus(2500);
        staff[0] = boss;
        staff[1] = new Employee("Çalışan 1", 0); // maaş altyazıda yok
        staff[2] = new Employee("Çalışan 2", 0); // maaş altyazıda yok

        for (Employee e : staff)
            System.out.println(e.getName() + " " + e.getSalary());

        Manager ikinci = new Manager("Yönetici 2", 8000, 2500);
        System.out.println(ikinci.getName() + " " + ikinci.getSalary());
    }
}
```

```
Yönetici 10500
Çalışan 1 0
Çalışan 2 0
Yönetici 2 10500
```

Yöneticinin maaşı adım adım şöyledir (ara adım eklendi, python ile hesaplandı):

- **Verilen:** `salary = 8000`, `bonus = 2500`.
- **İşlem:** `Manager.getSalary()` önce `super.getSalary()` ile `8000` değerini alır, sonra `bonus` ekler.
- **Sonuç:** $8000 + 2500 = 10500$.

13.3 Üç Yasak ve Düzeltmeleri

Hoca slaytın uyarılarını tek tek gösterdi:

- **`super(name)` ya da `super(salary)` yazmak:** Üst sınıfta böyle bir yapıcı tanımı olmadığı için hata verir. javac ile denendi: `Manager` yapıcısında bunlar `constructor Employee in class Employee cannot be applied to given types` hatası verdi.
- **`salary` ya da `super.salary` yazmak:** Alan private olduğu için hata verir. javac ile denendi: `salary has private access in Employee`. Alt sınıfta `name` için de aynı hata alındı.
- **`super.super` yazmak:** Böyle bir kullanım yoktur. javac ile denendi: `<identifier> expected`. Tek bir `super` yazılır, bir üst sınıfa çıkılır. Hoca "buradan dön, öbür `super`'e git" diye bir şey yapamayacağımızı söyledi.

  Ders sonrası not: Hoca "ne olursa olsun boş bir `super()` çalışır" dedi. Bu yalnız üst sınıfın parametresiz yapıcısı varsa geçerlidir. javac ile denendi: Parametresiz yapıcısı olan üst sınıfta önce üst sınıf, sonra alt sınıf yapıcısı çalıştı. Ama `Person` gibi yalnızca `Person(String)` tanımlı bir üst sınıfta `super(...)` yazılmayınca `constructor Person in class Person cannot be applied to given types` hatası çıktı. Yani bu örnekte `super(name)` zorunludur.

  Ders sonrası not: Hoca "`super`'i ilk satırda kullanmak gerekir" sözünü `super.getSalary()` üzerinde durduktan hemen sonra da söyledi. İlk satır kuralı yalnızca yapıcıdaki `super(...)` için anlatılır. javac ile denendi: Normal metotta `super.g()` ikinci satırda derlendi. Normal metot içinde `super("x")` yazmak ise hata verdi. Aynı yapıcıda iki kez `super(...)` da derlenmedi. Ayrıca kural Java sürümüne bağlıdır. javac ile denendi: JDK 25'te `this.s = s; super(n);` sırası derlenip çalıştı, `-source 21` ve `-source 17` ile `flexible constructors is not supported` hatası verdi. Dersin yapıldığı dönemin kuralı hocanın dediği gibidir.

13.4 main ve Sequence Diagram

`main` içinde `Employee` referanslarından oluşan üç elemanlı `staff` dizisi vardır. `boss` adında bir `Manager` nesnesi 8000 maaşla üretilir, `setBonus(2500)` çağrılır. Dizinin ilk elemanı `boss`'tur. İkinci eleman için ayrı bir referans atamaya gerek yoktur, yeni nesnenin referansı doğrudan yazılabilir: `new Employee(...)`. Üçüncü eleman da aynı biçimdedir. Dizide bir `Manager`, iki `Employee` vardır. `for-each` döngüsüyle her nesnenin `getName` ve `getSalary` sonucu ekrana yazılır.

Sequence diagram'ın anlatılan akışı şöyledir:

1. `main`, `Manager` nesnesi üretir, yanıt olarak `boss` referansı döner.
2. `main`, `boss.setBonus` çağrısını yapar.
3. `main`, iki `Employee` nesnesi üretir.
4. Döngünün her turunda sırayla birinci, ikinci ve üçüncü nesneye `getName` ve `getSalary` istekleri gider.

Hoca slayttaki diyagramda döngünün ayrı ayrı satırlarla açıldığını söyledi ve aynı diyagramı döngüyü kapsayan dikdörtgenle yeniden çizmeyi alıştırma olarak bıraktı.

14. Overriding ve Overloading Farkı

Bu iki kavram sık karıştırılır:

- **Overriding:** Kalıtımla yakından ilgilidir. Tanımlı bir metodu alt sınıfta aynı isimle yeniden tanımlayarak geçersiz kılmaktır. Yalnızca kalıtımda karşımıza çıkar. Örnek: `Manager` sınıfındaki `getSalary`.
- **Overloading:** Aynı adla, farklı parametrelerle birden çok metot tanımlamaktır. Kalıtımla ilgili değildir, sınıf tanımıyla ilgilidir. Örnek: `Manager` sınıfına ek bilgi alan ikinci bir yapıcı eklemek. Böylece `Manager`'ın iki yapıcısı olur.

  Ders sonrası not: Hoca "overloading association, aggregation ve composition ile ilgilidir" dedi. Doğrusu, overloading bu ilişkilerle ilgili değildir, hemen ardından söylediği gibi sınıf tanımıyla ilgilidir. Kalıtımda da kullanılabilir. javac ile denendi: Alt sınıfa `m(int)` ekleyince üst sınıfın `m()` metodu ile birlikte ikisi de çağrılabildi.

14.1 Private Metotlar ve final Sınıflar

Slayttaki bir cümle şöyle okundu: "Private methods of a super class can be overridden in its subclass." Devamında alt sınıfın orijinal sürüme `super` dahil hiçbir yolla erişemeyeceği söyleniyor.

> **Ders sonrası not:** Slaytın ilk cümlesi yanlıştır, private metotlar override edilemez. javac ile denendi: Üst sınıftaki private metodun üzerine `@Override` yazılınca `method does not override or implement a method from a supertype` hatası çıktı. Aynı adlı metot `@Override` olmadan yazılınca derlendi, ama üst sınıftan yapılan çağrıda yine üst sınıfın metodu çalıştı. Yani bu yeni, bağımsız bir metottur.

`final` olarak tanımlanan sınıf üst sınıf olarak kullanılamaz, örneğin `public final class Person`. javac ile denendi: `cannot inherit from final` hatası verir. C#'ta `final` yerine `sealed` kullanılır. Hoca kelimenin mühürlenmiş, damgalanmış anlamına geldiğini söyledi.

> **Hocanın vurgusu:** Sınıfa `final` yazarsanız kalıtım yapamazsınız. Metoda `final` yazarsanız override edemezsiniz. Dikkat edin.

15. Kalıtımın Yanlış Kullanımı: Hasta ve Doktor

Slaytta bir gereksinim belgesinden alıntı var. Basit bir uygulama için şu istekler sıralanıyor: Hastanın adı, T.C. kimlik numarası, doğum tarihi ve cep telefonu tutulacak. Aynı bilgiler doktorlar için de tutulacak. Ayrıca doktorun diploma numarası tutulacak.

İlk üç cümleyi okuyunca aklımıza şu tasarım gelebilir: Hastanın bütün bilgileri zaten doktorda var, doktorun bir de numarası var. İki sınıf yazmayalım, doktor hastadan kalıtım alsın. Hatta doktor da hasta olabilir, deriz. İlk bakışta olur gibi görünür. Ama bu yanlış bir modeldir, çünkü doktor bir hasta değildir. Kalıtımın özü olan is-a ilişkisi burada tutmaz.

Sonraki cümle işin yanlış olduğunu açıkça gösterir: Hastalara uygulanan tedaviler (treatments) de kaydedilecek. Hastaya yapılan tedavi doktora yapılmaz, doktor tedaviyi yapan kişidir. Her ortak alan bulduğumuzda biri alt sınıf, biri üst sınıf diyemeyiz.

Hoca is-a tutmayınca has ilişkisi aramanın da çözüm olmadığını ekledi. "Hocam is olmazsa has olur mu?" sorusu sık gelir. Doktor hastayı tedavi eder, ama bu alanlar üzerinden bir has ilişkisi kurmak da makul değildir.

15.1 Doğru Model

Bütün kişiler için genel bir kavram çıkarılır: `Person`.

- **`Person`:** Ad, kimlik numarası, cep telefonu ve doğum tarihi ortak alanlardır.
- **`Patient` ve `Doctor`:** `Person`'dan kalıtım alır, kendilerine özgü alan ve metotları vardır.
- **`Treatment`:** Ayrı bir sınıftır. Bir hastanın birden çok tedavisi olabilir. Hasta ile tedavi arasında association vardır. Kardinalite bir görününce aggregation da denebilir, ama parça-bütün olmadığı için association'dır.
- **`Treatment` - `Doctor`:** `Treatment` sınıfında `Doctor`'ı ya da onun metotlarını kullanabiliriz. Bu bir dependency'dir, tedavi doktora bağlıdır.

Hoca bu modelin yarı doğru (semi-correct) olduğunu, küçük bir alıntı olduğu için devamının da gelmesi gerektiğini belirtti. Tedavileri `Person`'a koymak da doğru olmaz. Tedavi hasta olan kişiler için yapılır, `Person`'a konursa doktoru da etkiler.

Model javac ile derlendi ve çalıştırıldı. Tarih ve kimlik alanlarında yer tutucu olarak - kullanıldı:

```
class Person {                  // ortak alanlar burada
    protected String name;
    protected String citizenId;
    protected String birthDate;
    protected String cellNumber;

    public Person(String name, String citizenId,
                  String birthDate, String cellNumber) {
        this.name = name;
        this.citizenId = citizenId;
        this.birthDate = birthDate;
        this.cellNumber = cellNumber;
    }
    public String getName() { return name; }
}

class Doctor extends Person {
    private String diplomaNo;

    public Doctor(String name, String citizenId, String birthDate,
                  String cellNumber, String diplomaNo) {
        super(name, citizenId, birthDate, cellNumber);
        this.diplomaNo = diplomaNo;
    }
}

class Treatment {
    private String description;

    public Treatment(String description) {
        this.description = description;
    }
    // Doctor alan değil, yalnız metot parametresi: dependency
    public void apply(Doctor doctor) {
        System.out.println(description + ", uygulayan: "
```

```
                + doctor.getName());
    }
}

class Patient extends Person {
    private Treatment[] treatments;   // doktora geçmez
    private int treatmentCount;

    public Patient(String name, String citizenId, String birthDate,
                   String cellNumber, int capacity) {
        super(name, citizenId, birthDate, cellNumber);
        treatments = new Treatment[capacity];
    }
    public void addTreatment(Treatment t) {
        treatments[treatmentCount] = t;
        treatmentCount++;
    }
    public int getTreatmentCount() { return treatmentCount; }
}

public class Klinik {
    public static void main(String[] args) {
        Doctor d = new Doctor("Doktor", "-", "-", "-", "-");
        Patient h = new Patient("Hasta", "-", "-", "-", 1);
        Treatment t = new Treatment("Tedavi");
        h.addTreatment(t);
        t.apply(d);
        System.out.println(h.getTreatmentCount());
    }
}
```

```
Tedavi, uygulayan: Doktor
1
```

> **Ders sonrası not:** Hoca slaytın "In the correct case, the treatments array is transferred to doctors" cümlesini okudu. Bağlama göre bu yanlış model için söyleniyor: Doktor hastadan kalıtım alırsa tedavi dizisi doktora da geçer.
>
> **Hocanın vurgusu:** En çok hata buradan yapılıyor. Doktorun bu alanları var, hastanın da bu alanları var deyip ortak gördüğünüz için kalıtım kurarsanız çok yanlış yaparsınız. En kolayı is-a ilişkisini kontrol etmektir. Buradan onay alamıyorsanız ona göre ilerleyin.

16. java.lang.Object ve toString()

`java.lang.Object` bütün sınıfların örtük üst sınıfıdır. Hoca buna slayttaki adıyla kozmik üst sınıf (cosmic superclass) dedi. Bu sınıfın bazı metotlarını kendi sınıflarınızda override edebilirsiniz. `@Override` notasyonu zorunlu değildir, ama metodun override edildiğini açıkça gösterir. javac ile denendi: Notasyon olmadan da alt sınıftaki `toString` çalıştı.

> **Ek bilgi (derste anlatılmadı):** `@Override`, yazım hatasını yakalamaya da yarar. javac ile denendi: `@Override` ile `tostring` yazılınca derleme hatası çıktı. Notasyon olmadan aynı metot derlendi, ama `Object`'in `toString`'i çalışmaya devam etti.

Örnek `toString()`'dir. Bu metodu daha önce kitap gibi örneklerde görmüştük, bir `String` döndürür. Dönüş tipi değiştirilemez, ama gövdesi yeniden tanımlanabilir. javac ile denendi: `public int toString()` yazmak `cannot override toString() in Object` hatası verdi. `introduceSelf` metodunda da benzer şekilde `this.getPlate()`, araya tırnakla metin ve `this.getYear()` birleştirilip `String` döndürülüyordu.

`toString` override etmenin avantajı, nesneyi doğrudan yazdırabilmektir:

```
class Car {
```

```
    private String plate;
    private int year;

    public Car(String plate, int year) {
        this.plate = plate;
        this.year = year;
    }
    public String getPlate() { return plate; }
    public int getYear() { return year; }

    @Override
    public String toString() {
        return this.getPlate() + " " + this.getYear();
    }
}

public class ToStringDeneme {
    public static void main(String[] args) {
        Car araba = new Car("PLAKA-1", 0);  // yıl: yer tutucu
        System.out.println(araba);          // toString() kendiliğinden
    }
}
```

```
PLAKA-1 0
```

Kod javac ile denendi. Plaka ve yıl altyazıda olmadığı için yer tutucudur.

Dersin Toparlaması

Hoca sonunda dönemin başından bu yana gelinen yolu özetledi: Nesnelerin özellikleri ve metotları vardır. Bu C'den farklıdır. Nesneler ya da sınıflar arasında beş ilişki türü vardır: association, dependency, aggregation, composition ve inheritance. Çok fazla anahtar kelime öğrenmedik, yeni olanlar `extends` ve `super`. Override kavramını, annotation olarak yazılan `@Override`'ı ve kalıtımın sonucu olan polymorphism'i gördük. Parça-bütün, is-a ve has ilişkileri iki haftadır konuşulanların genel kavramlarıdır.

> **Hocanın vurgusu:** Burada söylenen şeyler çabuk unutuluyor. Bu kavramları adınız gibi bilmeniz lazım. Size bir case study verildiğinde önce hangi sınıfların ve ilişkilerin olduğunu çıkarın, ondan sonra kodlamaya başlayın. Doğrudan kodlamaya başlarsanız sıkıntı çok büyük olur.

Hoca bu dersin algoritmik derinliğe çok girmediğini, `introduceSelf` gibi metotların kolay olduğunu, asıl beklentinin nesneye yönelik mantığı ve iskeleti ortaya çıkarmak olduğunu söyledi. Nesneye yönelik programlamanın kendine özgü bir jargonu vardır, ona hâkim olarak iş yapmak gerekir.

> **Hocanın vurgusu:** Kodunuzu derleye derleye gidin. Derleme hatası olan bir projeyi en son anda dışa aktarırsanız sınıflar yalnızca boş metotlarla çıkabilir. Bol bol kod yazın.

Sonraki adımlar da belirtildi: Kalıtım, polymorphism ve method overloading üzerine uygulamalı bir örnek yapılacak. Sonra abstract sınıflar görülecek, çoklu kalıtımın olup olamayacağı tartışılacak, oradan interface'e geçilecek. Ardından enumeration ve veri yapılarında gördüğünüz temel algoritmaların Java'daki karşılıkları gelecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.